

Programación en Móviles

LABORATORIO N° 07

Uso de Base de Datos SQLite con ROOM



DOCENTE:

**Juan León
Suiyon**

CURSO:

**Programación
en Móviles**

Laboratorio 07: Uso de Base de datos SQLite con ROOM

Caso: "GymTracker Pro" — App de Seguimiento de Entrenamiento

Contexto

Eres desarrollador de una app móvil para un gimnasio. Los socios necesitan registrarse, iniciar sesión y llevar control detallado de sus rutinas de ejercicio. La app debe funcionar offline usando **Room (SQLite)** y tener varias vistas conectadas mediante **Navigation Compose**.

Estructura de la Base de Datos

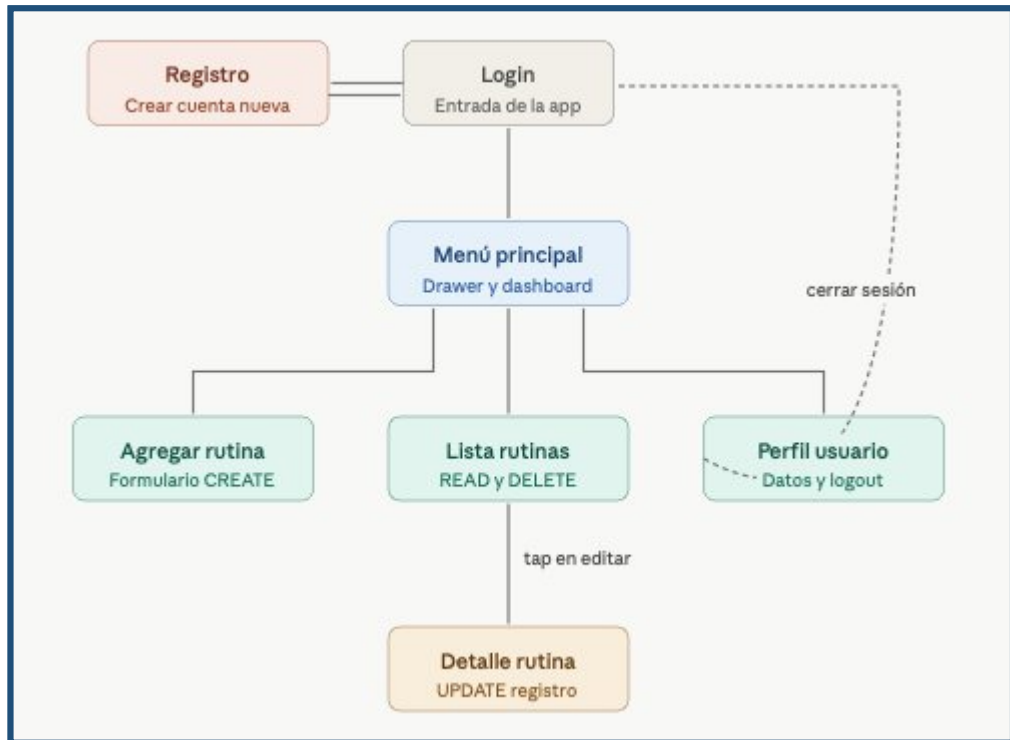
Tabla 1 : Usuario

```
@Entity
data class Usuario(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    @ColumnInfo(name = "nombre_usuario") val nombreUsuario: String,
    @ColumnInfo(name = "password") val password: String,
    @ColumnInfo(name = "nombre_completo") val nombreCompleto: String,
    @ColumnInfo(name = "email") val email: String,
    @ColumnInfo(name = "edad") val edad: Int,
    @ColumnInfo(name = "fecha_registro") val fechaRegistro: String
)
```

Tabla 2 : Rutina

```
@Entity
data class Rutina(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    @ColumnInfo(name = "usuario_id") val usuarioId: Int, //
    @ColumnInfo(name = "ejercicio") val ejercicio: String, //
    @ColumnInfo(name = "grupo_muscular") val grupoMuscular: String,
    @ColumnInfo(name = "series") val series: Int, //
    @ColumnInfo(name = "repeticiones") val repeticiones: Int, //
    @ColumnInfo(name = "peso_kg") val pesoKg: Double, //
    @ColumnInfo(name = "fecha") val fecha: String //
)
```

Diagrama del flujo de navegación

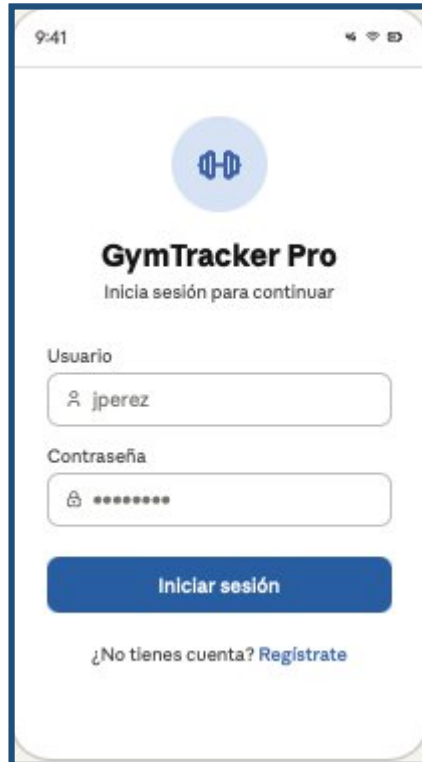


Las 6 vistas a desarrollar

1. **LoginScreen** — Autenticación con usuario y contraseña
2. **RegistroScreen** — Registro de nuevo usuario (alimenta la tabla Usuario)
3. **MenuPrincipalScreen** — Dashboard con navegación a las demás vistas (Scaffold + TopAppBar + Drawer)
4. **AgregarRutinaScreen** — Formulario para crear una rutina nueva (CREATE)
5. **ListaRutinasScreen** — Listado en LazyColumn de las rutinas del usuario logueado (READ + DELETE)
6. **DetalleRutinaScreen** — Editar una rutina existente (UPDATE)
7. **PerfilUsuarioScreen** — Información del usuario y botón de cerrar sesión

Login Screen

Lógica: Consulta `usuarioDao.buscarPorCredenciales(user, pass)`. Si retorna un `Usuario`, navega a `MenuPrincipal` pasando el `usuarioId`; si es `null`, muestra un `Snackbar` de error



The login screen for GymTracker Pro features a blue header bar with the time 9:41 and status icons. Below the header is a blue circle containing a white dumbbell icon. The app name "GymTracker Pro" is displayed in bold, followed by the text "Inicia sesión para continuar". The login form consists of two input fields: "Usuario" with the text "jperez" and "Contraseña" with masked characters "*****". A blue button labeled "Iniciar sesión" is positioned below the password field. At the bottom, there is a link that reads "¿No tienes cuenta? Regístrate".

Registro Screen

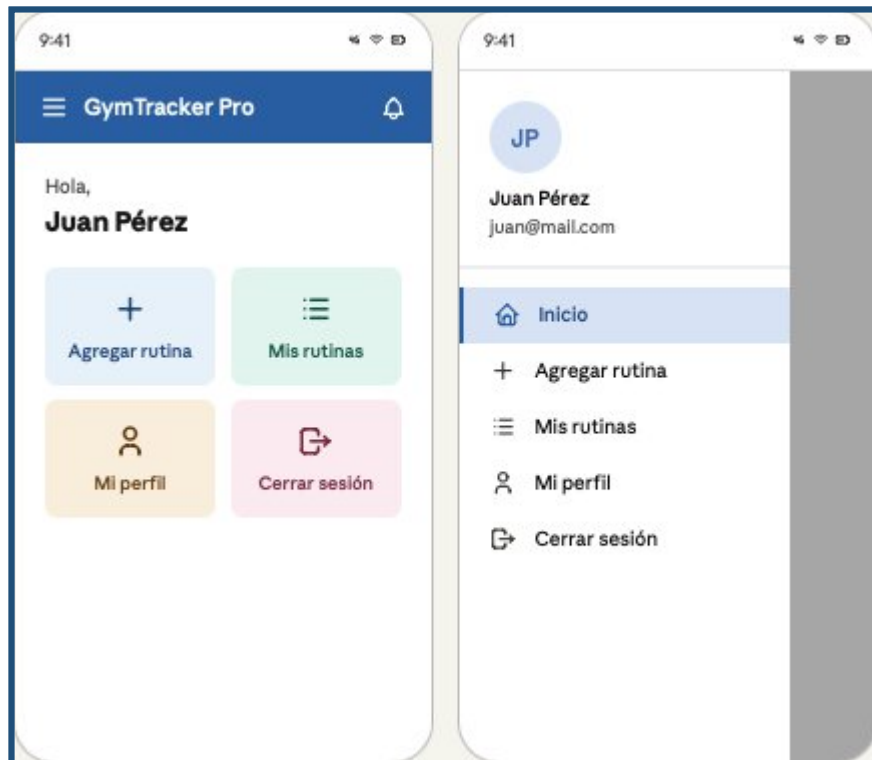
Lógica: Valida que el email tenga formato válido y que el usuario no exista. Llama `usuarioDao.insertar(usuario)` y regresa al Login con `navController.popBackStack()`



The registration screen for GymTracker Pro has a blue header bar with the time 9:41 and status icons. A blue bar below the header contains a white back arrow and the text "Crear cuenta". The registration form includes five input fields: "Nombre completo" with "Juan Pérez Vela", "Usuario" with "jperez", "Email" with "juan@mail.com", "Edad" with "25", and "Contraseña" with masked characters "*****". A blue button labeled "Registrarme" is located at the bottom of the form.

Menu Principal (con Drawer y TopAppBar)

Lógica: Aquí usas ModalNavigationDrawer + Scaffold + TopAppBar. El ícono de menú abre el drawer; las tarjetas y los ítems del drawer llaman a `navController.navigate(ruta)`.



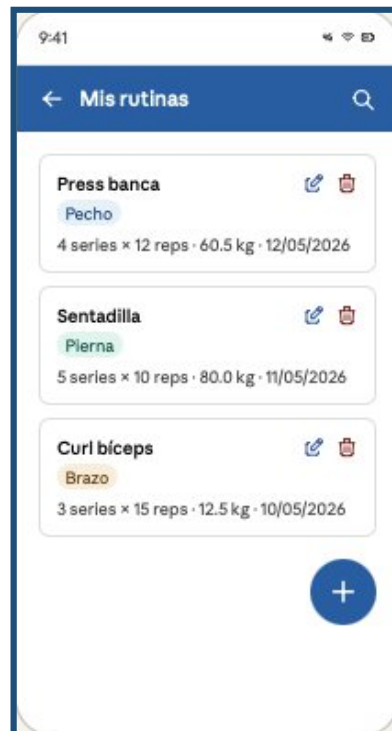
Agregar rutina(CREATE)

Lógica: Construye un objeto Rutina con el `usuarioid` que viene del Login (pasado como argumento de navegación) y llama a `rutinaDao.insertar(rutina)`. Luego `navController.popBackStack()`.

The image shows a screenshot of a mobile application form titled 'Nueva rutina'. The form has a blue header bar with a back arrow and the title. The form fields are: 'Ejercicio' with a text input containing 'Press banca'; 'Grupo muscular' with a dropdown menu showing 'Pecho'; 'Series' with a text input containing '4'; 'Repeticiones' with a text input containing '12'; 'Peso (kg)' with a text input containing '60.5'; and 'Fecha' with a date picker showing '12/05/2026'. At the bottom of the form is a blue button labeled 'Guardar rutina'.

Lista de Rutinas (READ + DELETE)

Lógica: LazyColumn que recorre rutinaDao.listarPorUsuario(usuarioId). El ícono de lápiz navega a DetalleRutinaScreen/{rutinaId}; el de basura llama a rutinaDao.eliminar(rutina) con un AlertDialog de confirmación. El **FAB** flotante navega a AgregarRutina.



Detalle/Editar Rutina(UPDATE)

Lógica: Recibe el rutinaId por argumento de navegación. En LaunchedEffect(Unit) carga rutinaDao.buscarPorId(id) y llena los campos. Al guardar llama rutinaDao.actualizar(rutina.copy(...)). El ícono de basura en el TopAppBar también permite eliminar desde aquí.



Perfil del Usuario

Lógica: Carga `usuarioDao.buscarPorId(usuarioId)` y muestra estadísticas calculadas con `@Query` agregadas como `SELECT COUNT(*) FROM Rutina WHERE usuario_id = :id` y `SELECT SUM(peso_kg * series * repeticiones) FROM Rutina WHERE usuario_id = :id`. El botón rojo limpia el estado y navega a Login con `popUpTo("login") { inclusive = true }`.



¿Por dónde empezar?

Te recomiendo este orden:

1. **Tablas + DAOs primero** (Usuario y Rutina) — sin UI todavía, solo verifica en Database Inspector.
2. **Login y Registro** — son las más simples y te dan el `usuarioId` que el resto necesita.
3. **NavHost completo** con pantallas vacías — comprueba que la navegación funciona antes de meter lógica.
4. **Menú con Drawer** + las 3 pantallas CRUD encima.
5. **Perfil** al final (es la más sencilla).

Nota : Esta aplicación puede ser realizada por dos participantes, tengan presente de que la evaluación es individual , eso quiere decir que cada uno tiene que levantar la aplicacion en su maquina, para que realice las modificaciones en el codigo y las preguntas que realizara el instructor.